Project Title: Clean Bengaluru – Smart Waste Management & Automated Escalation System

1. Overview

Clean Bengaluru is a smart accountability platform designed to empower citizens to participate seamlessly in urban cleanliness. The system allows users to report uncollected garbage issues by dropping a map pin, capturing a "Before" image, and providing localized context. Leveraging reverse-geocoding and automated background scheduling, the platform tracks the severity of aging reports, identifies recurring hotspots, and delivers actionable, escalating email alerts directly to municipal authorities to ensure optimized routing and guaranteed resolution.

2. Objective

To build a scalable, modular, and secure tracking platform that automates the categorization and escalation of urban waste reports. By converting unstructured citizen inputs into structured, time-tracked tickets, the system streamlines municipal response times, enforces community accountability through a strict "Proof-of-Work" (After-photo) validation, and utilizes comprehensive log analysis to pinpoint delays and optimize long-term city resource allocation.

3. Data Processing & Analysis

Supported Inputs

- **Visual Data**: High-resolution "Before" and "After" images (.JPG, .PNG) captured by users.
- **Location Data**: Geographic coordinates (Latitude & Longitude) and automatically reverse-geocoded Area/Locality names via Google Maps APIs.
- **Contextual Data**: User-provided textual descriptions detailing the waste environment and severity.
- **System Logs**: Application and server-level logs generated during report submission, chronological state transitions, and scheduled automated escalations.

Data Characteristics

- **Dynamic & Real-Time**: Reports are generated asynchronously by users and instantly broadcasted to a live geographic map.
- **Unstructured to Structured**: Raw user uploads and coordinates are standardized and transformed into parsable JSON payloads for the database.
- **Log Heavy**: Every API interaction, manual resolution, and background status transition generates chronological logs for performance auditing.

Analysis Requirements

- Monitor time-to-decay metrics to automatically classify pending reports based on their age.
- Parse location grids to aggregate area-wide reports, identifying high-density waste zones.
- Analyze system logs and email-dispatch footprints to track compliance latency and identify bottlenecks in municipal response.

4. Processing Engine

System Flow

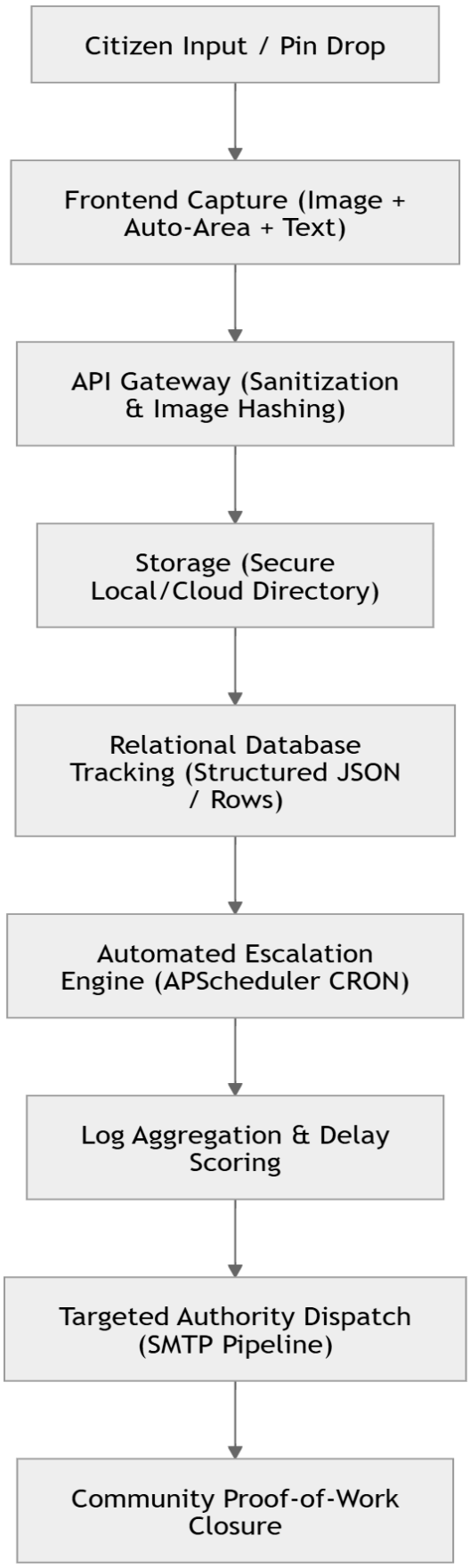
1. **Ingestion**: Raw image, exact GPS coordinates, auto-detected area, and text are submitted via the frontend API.
2. **Reverse Geocoding**: Coordinates are translated into human-readable localities on the client side to categorize the data geographically.
3. **Tracking Initialization**: The backend stores the payload in the relational database, assigning an immutable timestamp and a default "Pending" status.
4. **Automated Escalation (CRON)**: A daily background scheduler scans active reports and calculates "days pending" to route escalating warnings.
5. **Proof-of-Work Resolution**: Community members or workers upload an "After" photo, verified by the system to permanently close the ticket and visually update the public map.

Automated Escalation & Severity Levels

- **Standard (Level 1 - Day 2)**: Routine uncollected status. A primary reminder email is automatically generated and dispatched to local authorities.
- **High (Level 2 - Day 4)**: Persistent waste pile. A follow-up warning is routed to the primary dashboard.
- **Critical (Level 3 - Day 7+)**: Long-term hazard. Maximum escalation protocols are triggered, looping in senior operational (SPCOM) emails with full coordinate and imagery logs.

5. End-to-End Processing Flow

The below diagram represents the end-to-end workflow of the Clean Bengaluru system. A citizen reports a waste issue by dropping a pin and uploading an image with details. The frontend captures the data, which is then validated and sanitized through the API gateway. The data is securely stored and tracked in a database. A background scheduler monitors pending reports and triggers time-based escalation. Logs are analyzed to measure delays, and alerts are sent to authorities via email. Finally, the issue is closed only after a verified “after” image is uploaded as proof-of-work.


Citizen Input / Pin Drop
Frontend Capture (Image + Auto-Area + Text)
API Gateway (Sanitization & Image Hashing)
Storage (Secure Local/Cloud Directory)
Relational Database Tracking (Structured JSON / Rows)
Automated Escalation Engine (APScheduler CRON)
Log Aggregation & Delay Scoring
Targeted Authority Dispatch (SMTP Pipeline)
Community Proof-of-Work Closure

6. API Design

Endpoint

POST /api/v1/reports

Request JSON


```
{
  "location": {
    "lat": 12.9716,
    "lng": 77.5946,
    "area": "Indiranagar"
  },
  "before_image": "[binary file]",
  "description": "Garbage pile blocking roadside drainage",
  "timestamp": "2026-03-26T10:30:00Z"
}
```


Response JSON


```
{
  "report_id": 1025,
  "location": {
    "lat": 12.9716,
    "lng": 77.5946,
    "area": "Indiranagar"
  },
  "status": "Pending",
  "created_at": "2026-03-26T10:30:02Z",
  "notification_level": 0
}
```

7. Frontend Requirements

- **Responsive Web App**: Built with modern web tooling (HTML/CSS/JS) natively integrating camera access and dynamic GPS prompting without heavy framework bloat.
- **Interactive Map Dashboard**: Real-time tracking of active, unresolved hotspots (Red Pins) and fully resolved areas (Green Pins) utilizing Google 'AdvancedMarkerElements'.
- **Security**: Client-side payload sanitization and strict MIME-type image validation before transmission.
- **Accessibility & UX**: Premium "glassmorphism" user interface featuring clear "Do's and Don'ts" guidelines, seamless table tracking, and high-contrast usability suited for on-the-go outdoor reporting.

8. Log Analysis

The system generates HTTP access logs that capture all incoming API requests. These logs are used to monitor application behavior, debug issues, and verify system performance.

Each log entry contains:

- Client IP address
- Timestamp
- HTTP method (GET, POST)
- API endpoint
- Response status code

Sample Log Entries

```
127.0.0.1 - - [26/Mar/2026 22:07:03] "GET /api/reports HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 22:07:26] "POST /api/reports HTTP/1.1" 201 -
127.0.0.1 - - [26/Mar/2026 22:07:29] "GET /api/reports HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 22:07:49] "POST /api/reports/3/resolve HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 22:07:50] "GET /api/reports HTTP/1.1" 200 -
```

Analysis of Logs

1. Request Flow

The logs indicate a typical user interaction sequence:

- A GET request retrieves existing reports
- A POST request creates a new report (201 Created)
- Another GET request confirms the updated data
- A POST request resolves a specific report

- A final GET request retrieves the updated list

  This confirms that the system correctly implements CRUD operations.

---

2. Status Code Analysis

- 200 OK indicates successful request handling
- 201 Created indicates successful resource creation

  No error codes (4xx or 5xx) were observed, indicating stable system performance during execution.

---

3. Performance Observation

- Time gaps between requests are minimal (approximately 1–3 seconds)
- No delays or failures detected
- Indicates smooth backend response and proper frontend interaction

---

4. System Behavior Insights

- The frontend refreshes data after each operation
- The backend correctly processes creation and resolution actions
- API endpoints follow RESTful design principles

9. Example Use Case

Scenario:
A resident in Indiranagar notices an overflowing garbage bin on the roadside.

Action:
The user opens the Clean Bengaluru web app, drops a map pin, captures a “Before” image, and submits the report. The system automatically records GPS coordinates and identifies the area via reverse geocoding.

Processing:
The report is stored with a timestamp and marked as “Pending.” After 48 hours without resolution, the system triggers a **Level 1 escalation email** to municipal authorities. If the issue persists, further escalations are triggered at Day 4 and Day 7.

Resolution:
A sanitation worker cleans the area and uploads an “After” image. The system verifies the

proof-of-work, updates the report status to “Resolved,” stops further escalations, and updates the map marker from red to green.